

Dokumen Operasional Deployment

Dokumen ini merangkum proses *deployment* aplikasi ke lingkungan *cluster* Google Kubernetes Engine (GKE), baik melalui metode manual maupun otomatis dengan CI/CD.

- **Kubernetes (k8s)** adalah platform *open-source* untuk mengelola beban kerja dan layanan dalam kontainer.
- Memudahkan untuk mengatur banyak kontainer di banyak *host*, menskalakannya sebagai layanan mikro (*microservices*), dan melakukan *deployment rollout*.
- Merupakan kumpulan API untuk men-*deploy* kontainer pada sekelompok *node* yang disebut *cluster*.
- Terbagi menjadi serangkaian komponen utama yang berjalan sebagai *control plane* dan serangkaian *node* yang menjalankan kontainer.

Google Kubernetes Engine (GKE) dapat men-*deploy*, mengelola, dan menskalakan Kubernetes. Berikut adalah beberapa fitur GKE: Dikelola sepenuhnya (*fully managed*), sistem operasi dioptimalkan untuk skala cepat, GKE Autopilot mengelola konfigurasi *cluster*, pembaruan otomatis untuk memastikan *cluster* memiliki versi Kubernetes stabil terbaru, perbaikan otomatis pada *node* yang sehat, dapat menskalakan *cluster* itu sendiri, serta terintegrasi dengan Cloud Build, Google Cloud Observability, dan Virtual Private Clouds (VPC).

1. Deployment Manual

Deployment manual dilakukan untuk kebutuhan *testing* atau *deployment* awal. Proses ini melibatkan serangkaian perintah baris yang dieksekusi secara berurutan.

Langkah-langkah Deployment

Sebelum melakukan tahapan deployment, login gcloud terlebih dahulu.

```
gcloud auth login
```

Berikut adalah tahapan manual untuk men-*deploy* aplikasi ke GKE:

Langkah 0: Inisiasi Cluster GKE dengan Terraform

Tahap ini hanya dilakukan sekali untuk membuat atau memperbarui infrastruktur *cluster*.

1. Inisiasi Terraform

```
# Lakukan di folder `tf/`  
terraform init
```

2. Buat Rencana Eksekusi

```
terraform plan -out tfplan
```

3. Terapkan Konfigurasi

```
terraform apply -auto-approve tfplan
```

4. Konfigurasi `kubectl`

```
# Lihat konteks saat ini
kubectl config current-context --project primeval-rune-467212-t9

# Ubah konteks kubectl agar berinteraksi dengan cluster GKE
gcloud container clusters get-credentials wondr-desktop-cluster --zone asia-southeast1-a --project primeval-rune-467212-t9
```

Langkah 1: Build dan Push Docker Image

Setelah infrastruktur siap, *image* Docker untuk *frontend* dan *backend* dibangun dan di-*push* ke Google Container Registry (GCR).

Catatan Penting: Pastikan file Firebase JSON sudah tersedia di direktori *frontend* dan *backend* sebelum proses *build*.

a. Frontend

- **Build Image:** Gunakan `--build-arg` untuk menyisipkan *environment variable* yang tidak bisa diatur melalui `.yaml` Kubernetes.

```
# Lakukan di root folder aplikasi
docker build \
  --build-arg VITE_BACKEND_BASE_URL=GANTISAYA \
  --build-arg VITE_VERIFICATOR_BASE_URL=[https://verificator-secure-onboarding-system-441501015598.asia-southeast1.run.app](https://verificator-secure-onboarding-system-441501015598.asia-southeast1.run.app) \
  --build-arg VITE_FIREBASE_API_KEY=AIzaSyCTXgqBktnmUo8z5VkxMuwBpLkBGZ_syj0 \
  ...
-t gcr.io/model-parsec-465503-p3/frontend-secure-onboarding-system:latest
./frontend-secure-onboarding-system
```

- **Push Image:**

```
docker push gcr.io/model-parsec-465503-p3/frontend-secure-onboarding-system:latest
```

b. Backend

- **Build Image:**

```
docker build -t gcr.io/model-parsec-465503-p3/backend-secure-onboarding-system:latest ./backend-secure-onboarding-system
```

- **Push Image:**

```
docker push gcr.io/model-parsec-465503-p3/backend-secure-onboarding-system:latest
```

Langkah 2: Apply Manifest Kubernetes

Manifest Kubernetes (`.yaml` files) diterapkan untuk membuat *deployment*, *service*, dan konfigurasi lainnya di *cluster* GKE.

- **Database:**

```
kubectl apply -f ./k8s/application/postgresql-deployment.yaml
kubectl apply -f ./k8s/application/postgresql-secrets.yaml
kubectl apply -f ./k8s/application/postgresql-service.yaml
```

- **Backend:**

```
kubectl apply -f ./k8s/application/app-secrets.yaml
kubectl apply -f ./k8s/application/backend-configmaps.yaml
kubectl apply -f ./k8s/application/backend-deployment.yaml
kubectl apply -f ./k8s/application/backend-networkpolicy.yaml
```

- **Frontend:**

```
kubectl apply -f ./k8s/application/frontend-certificate.yaml
kubectl apply -f ./k8s/application/frontend-configmap.yaml
kubectl apply -f ./k8s/application/frontend-deployment.yaml
kubectl apply -f ./k8s/application/frontend-service.yaml
```

- **Network (Ingress):**

```
kubectl delete -f ./k8s/application/ingress.yaml
kubectl apply -f ./k8s/application/ingress.yaml
```

- **Verifikasi Deployment:**

```
kubectl describe pods [nama_pods]
```

2. Deployment Otomatis dengan CI/CD

Pipeline CI/CD diimplementasikan menggunakan Jenkins untuk mengotomatisasi seluruh alur kerja, dari *checkout* kode hingga *deployment* di lingkungan produksi.

Visualisasi Pipeline

Berikut adalah gambaran umum dari alur kerja CI/CD menggunakan Jenkins Declarative Pipeline:



Tahapan dalam Pipeline

1. **Source Code Checkout:** Mengambil kode sumber dari repositori *backend*, *frontend*, dan konfigurasi ops. Tahap ini diparalelkan untuk efisiensi.
2. **Inject Secrets:** Menginjeksikan kunci rahasia (seperti Firebase Private Key) dari Jenkins Credentials ke direktori proyek.
3. **Build Application:** Membangun aplikasi *backend* (Maven) dan *frontend* (Node.js) secara paralel.
4. **Testing & Quality Analysis:** Menjalankan pengujian unit, analisis kualitas kode, dan pemindaian keamanan (misalnya dengan SonarQube atau OWASP ZAP) secara paralel.
5. **Package Application:** Mengemas aplikasi yang sudah teruji.
6. **Docker Build & Registry:** Membangun *image* Docker untuk *backend* dan *frontend*, lalu mendorongnya ke Google Container Registry (GCR) secara paralel.
7. **Staging Deployment:** Menerapkan *image* Docker ke lingkungan *staging* untuk pengujian integrasi.
8. **Staging Tests:** Menjalankan tes integrasi otomatis di lingkungan *staging*.
9. **QA Approval:** Membutuhkan persetujuan manual dari tim QA sebelum melanjutkan ke produksi.
10. **Production Deployment:** Menerapkan manifest Kubernetes (*.yaml*) ke *cluster* GKE produksi.
11. **Production Verification:** Melakukan verifikasi pasca-deployment untuk memastikan layanan berjalan dengan baik.
12. **Post-Deployment Report:** Menghasilkan laporan akhir dari proses deployment.

Konfigurasi Jenkins

- **Lingkungan Jenkins:** Menggunakan Docker-in-Docker (*dind*) untuk menjalankan Jenkins di dalam container, yang memiliki akses ke Docker socket *host*.
- **Plugins:** Menggunakan plugin penting seperti Git, Docker Pipeline, NodeJS, dan JDK Tool.
- **Secrets Management:** Mengelola kredensial sensitif (GCR Service Account Key, Firebase Private Key) menggunakan fitur **Jenkins Credentials** sebagai *secret file* atau *secret text*.

Troubleshooting Umum

- **Ruang Disk Penuh:** Jika proses *build* Jenkins terhenti, periksa penggunaan disk VM. Solusinya adalah meningkatkan ukuran disk VM di Google Cloud.
- **Izin Akses:** Jika terjadi *permission denied*, pastikan user Jenkins memiliki izin yang memadai (misalnya, dengan menggunakan `chmod`) pada direktori yang relevan.
- **IP Dinamis:** Pastikan alamat IP eksternal VM Jenkins diubah menjadi statis untuk menghindari masalah konektivitas.

Catatan

- **Urutan Ingress:** Dalam konfigurasi *ingress*, urutan *path* sangat penting. *Path* yang paling spesifik, seperti `/api/auth`, harus diletakkan di atas *path* yang lebih umum, seperti `/`, untuk memastikan *routing* berjalan dengan benar.
- **Build History:** Dokumentasikan setiap riwayat *build* di Jenkins (sukses atau gagal) dengan deskripsi yang jelas untuk mempermudah proses *debugging* dan audit.